Informatik 1: Computertechnik

Wirtschaftsingenieurwesen Bachelor

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Vom Schalter zum Computer: Transistor und Logik
2. Bit, Byte und Zahlensysteme
3. EVA vertieft: Die von-Neumann-Architektur
4. Die Speicherhierarchie
5. Das Betriebssystem

Heute wird nicht programmiert – wir schauen in die Maschine. Alles, was Sie ab nächster Woche in Python schreiben, landet am Ende in dieser Hardware.

TL;DL – Kurzfassung

- **Alles ist binär:** Computer kennen nur zwei Zustände – an/aus, 1/0. Zahlen, Texte, Bilder sind Bitmuster.
- **von Neumann:** Programm und Daten liegen im **selben Speicher**; die CPU holt, dekodiert und führt Befehle aus.
- **Speicherhierarchie:** schnell-klein-teuer (Register) bis langsam-groß-billig (Festplatte, Cloud).
- **Betriebssystem:** verwaltet Prozesse, Speicher, Dateien und Geräte – die Schicht zwischen Ihrem Programm und der Hardware.

Warum Computertechnik?

Zwei Fragen, die Sie nach heute beantworten können

1. **Laptop-Kauf:** Was bedeuten “3,2 GHz, 8 Kerne, 16 GB RAM, 512 GB SSD” – und wofür zahlt man da eigentlich?
2. **Veggie Soles**, unser veganer Sneaker-Shop: Eine Kundin klickt auf *Bestellen*. **Was passiert dabei im Rechner** – vom Klick bis zur gespeicherten Bestellung?

> Die zweite Frage begleitet uns das ganze Semester: heute die Hardware, ab nächster Woche die Programme darauf.

1. Der Transistor: ein elektrischer Schalter

- Ein **Transistor** lässt Strom durch oder sperrt ihn: **an oder aus**
- Zwei Zustände genügen: **an = 1, aus = 0**
- Moderne CPUs enthalten **Milliarden** Transistoren auf wenigen cm^2

Deshalb ist alles im Computer **binär**: Es gibt keine “halben” Schalter. Zwei Zustände sind billig, robust und schnell zu unterscheiden.

1. Vom Transistor zum Logikgatter

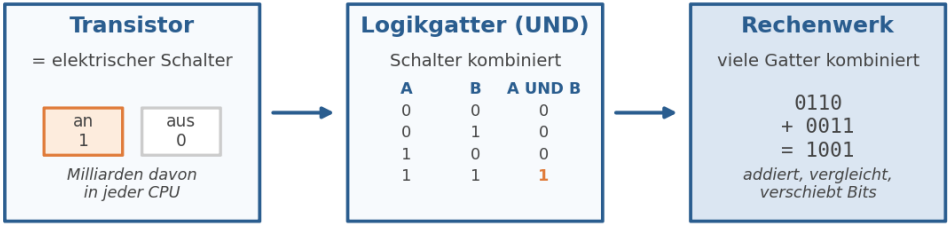


Abbildung 1: Vom Schalter zum Rechenwerk

1. Vom Gatter zum Rechenwerk

- Ein **Logikgatter** kombiniert Schalter: UND, ODER, NICHT
- **UND-Gatter:** Ausgang ist nur 1, wenn *beide* Eingänge 1 sind
- Viele Gatter zusammen bilden **Schaltungen, die rechnen** – z. B. einen Addierer für Binärzahlen
- Mehr Gatter → Vergleichen, Verschieben, Multiplizieren … eine komplette **ALU**

Merken: Rechnen ist im Kern nichts anderes als geschickt verdrahtete an/aus-Schalter. Die Logik dahinter (`und`/`oder`/`nicht`) sehen Sie in Python wieder – Foliensatz 08.

2. Bit und Byte

- **1 Bit** = ein Schalter = 0 oder 1
- **1 Byte** = 8 Bit = 2^8 = **256 Zustände**

Größenordnungen im Alltag

Einheit	Umfang	Beispiel
1 KB (Kilobyte)	~1.000 Byte	eine halbe Textseite
1 MB (Megabyte)	~1 Mio. Byte	ein Foto (komprimiert)
1 GB (Gigabyte)	~1 Mrd. Byte	~250 Musiktitel
1 TB (Terabyte)	~1 Bio. Byte	komplette Produkt-Datenbank von Veggie Soles – viele tausend Mal

2. Stellenwertsysteme: Dezimal neu betrachtet

Sie rechnen seit der Grundschule in einem **Stellenwertsystem** – zur Basis 10:

Stelle	4	7	1	1
Stellenwert	1000	100	10	1
Beitrag	4000	700	10	1

4711 = 4000 + 700 + 10 + 1

- Jede Stelle hat einen **Stellenwert** (Potenz der Basis 10)
- Die Ziffer sagt, **wie oft** dieser Stellenwert zählt

Das Binärsystem funktioniert **genauso** – nur mit Basis 2 und den Ziffern 0 und 1.

2. Binär nach Dezimal

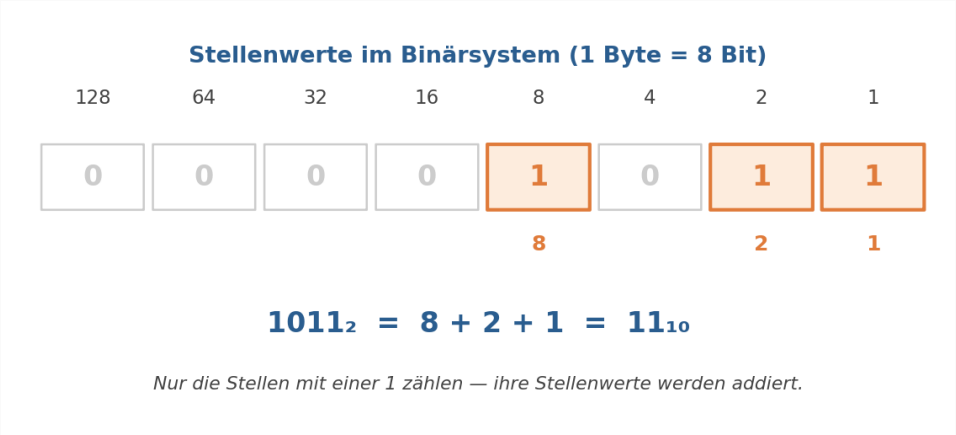


Abbildung 2: Stellenwerte im Binärsystem

2. Dezimal nach Binär: Restwertmethode

Beispiel: 25 in Binärdarstellung

Rechnung	Ergebnis	Rest
25 : 2	12	**1**
12 : 2	6	**0**
6 : 2	3	**0**
3 : 2	1	**1**
1 : 2	0	**1**

Reste **von unten nach oben** lesen: $25_{10} = 11001_2$

Probe: $16 + 8 + 1 = 25$ ✓

2. Sofort ausprobieren: Umrechnen von Hand

✎ **Sofort ausprobieren** (3 Minuten, Stift und Papier):

1. Was ist 1101_2 im Dezimalsystem?
2. Stellen Sie 19_{10} binär dar (Restwertmethode).
3. Was ist die **größte** Zahl, die in ein Byte (8 Bit) passt?

Vergleichen Sie mit Ihrer Sitznachbarin / Ihrem Sitznachbarn.

2. Hexadezimal: die Abkürzung

- Binärzahlen werden schnell **lang und unleserlich**: 11111111_2
- **Hexadezimal** (Basis 16) fasst je **4 Bit in eine Ziffer**: 0–9, dann A–F
- $11111111_2 = \mathrm{FF}_{16} = 255_{10}$

Wo Ihnen Hex begegnet

Anwendung	Beispiel
Farbcodes im Web	`#FF6600` (Orange)
MAC-Adressen	`A4:83:E7:2B:11:09`
Fehlercodes / Speicheradressen	`0x0000007B`

2. Live-Demo: Python als Taschenrechner

▶ **Live-Demo** – ich tippe, Sie schauen zu. *Python lernen wir ab nächster Woche; heute benutze ich es nur als Taschenrechner:*

```
bin(11)          # '0b1011'   -> binär
hex(255)         # '0xff'     -> hexadezimal
int("1011", 2)   # 11         -> zurück ins Dezimalsystem
2 ** 10          # 1024       -> warum 1 "Kilo"byte oft 1024 ist
```

Der Computer rechnet **immer** binär – Dezimal, Hex & Co. sind nur Anzeigeformate für Menschen.

2. Was steckt in einem Byte? Alles.

Dieselben 8 Bit können bedeuten:

Interpretation	`01000001` ist …
Zahl	65
Zeichen	`"A"`
Teil eines Bildes	ein Pixel-Helligkeitswert
Teil eines Befehls	eine CPU-Instruktion

Erst der Kontext gibt Bits eine Bedeutung. Wie aus Zahlen Buchstaben werden (ASCII, Unicode), vertiefen wir bei den Datentypen – Foliensatz 05.

3. EVA vertieft: von der Idee zur Hardware

Aus E2 kennen Sie das **EVA-Prinzip**. Heute mit Hardware-Brille:

EVA-Schritt	Hardware	Veggie-Soles-Bestellung
Eingabe	Tastatur, Maus, Touch, Netzwerk	Kundin klickt “Bestellen”
Verarbeitung	CPU (+ RAM als Arbeitsfläche)	Preis berechnen, Lager prüfen
Ausgabe	Monitor, Drucker, Netzwerk	Bestellbestätigung anzeigen
Speicher	SSD, Festplatte	Bestellung dauerhaft sichern

3. Die von-Neumann-Architektur

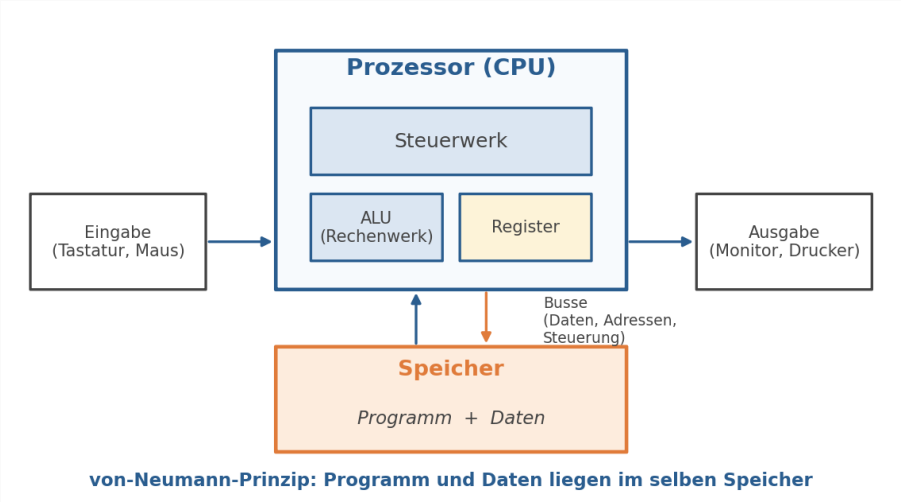


Abbildung 3: von-Neumann-Architektur

3. von Neumann: die zentrale Idee

- Entwurf von **John von Neumann (1945)** – bis heute die Grundarchitektur fast aller Computer
- **Programm und Daten liegen im selben Speicher**
- Konsequenz: Programme sind **austauschbar wie Daten** – derselbe Rechner spielt Musik, rechnet Lohnbuchhaltung und verkauft Sneaker

Vorher waren Maschinen **fest verdrahtet** für eine Aufgabe. Der von-Neumann-Rechner ist eine **Universalmaschine** – deshalb lohnt es sich, Programmieren zu lernen.

3. Die CPU von innen

Komponente	Rolle	Eselsbrücke
Steuerwerk	holt Befehle, steuert den Ablauf	der Dirigent
ALU (Rechenwerk)	rechnet und vergleicht	der Taschenrechner
Register	halten die gerade benötigten Werte	der Notizzettel

- Alle drei arbeiten im Takt zusammen – **Milliarden Mal pro Sekunde**

3. Der Befehlszyklus: Holen – Dekodieren – Ausführen

Beispiel: “Addiere Preis und Versandkosten”

1. **Holen (Fetch):** Steuerwerk holt den nächsten Befehl aus dem Speicher
2. **Dekodieren (Decode):** Was ist zu tun? *Addieren!* Welche Werte? *Preis, Versandkosten*
3. **Ausführen (Execute):** ALU addiert; Ergebnis landet in einem Register

... und von vorn, für den nächsten Befehl.

Jedes Python-Programm wird am Ende zu **Tausenden solcher Mini-Schritte**. Der Computer ist nicht klug – er ist *schnell*.

3. Takt und Kerne: Was heißt “3,2 GHz, 8 Kerne”?

- **Takt (GHz):** Schrittfrequenz der CPU – 3,2 GHz = **3,2 Milliarden Takte pro Sekunde**
- Höherer Takt = mehr Wärme und Stromverbrauch → physikalische Grenze
- Ausweg: **mehrere Kerne** – 8 Kerne = 8 CPUs auf einem Chip, die parallel arbeiten
- Parallel hilft nur, wenn sich die Arbeit **aufteilen** lässt (viele Bestellungen gleichzeitig: ja; eine einzelne Rechnung: kaum)

3. Busse: die Datenautobahnen

Bus	Transportiert	Frage
Datenbus	die eigentlichen Werte	*Was?*
Adressbus	woher/wohin im Speicher	*Wo?*
Steuerbus	Kommandos (lesen/schreiben)	*Wie?*

- Verbinden CPU, Speicher und Geräte
- Auch der schnellste Prozessor wartet, wenn der Bus verstopft ist – ein Grund, warum **Speicherzugriff** oft der Flaschenhals ist

4. Warum eine Speicherhierarchie?

Der Zielkonflikt: Speicher soll **schnell**, **groß** und **billig** sein – **alle drei zusammen gibt es nicht.**

Speicher	typischer Zugriff	Vergleich
Register	< 1 ns	Gedanke
RAM	~100 ns	Griff ins Regal
SSD	~0,1 ms	Gang in den Keller
Festplatte	~10 ms	Fahrt ins Außenlager

Lösung: **Stufen kombinieren** – wenig Superschnelles nah an der CPU, viel Billiges weiter weg.

4. Die Speicherpyramide

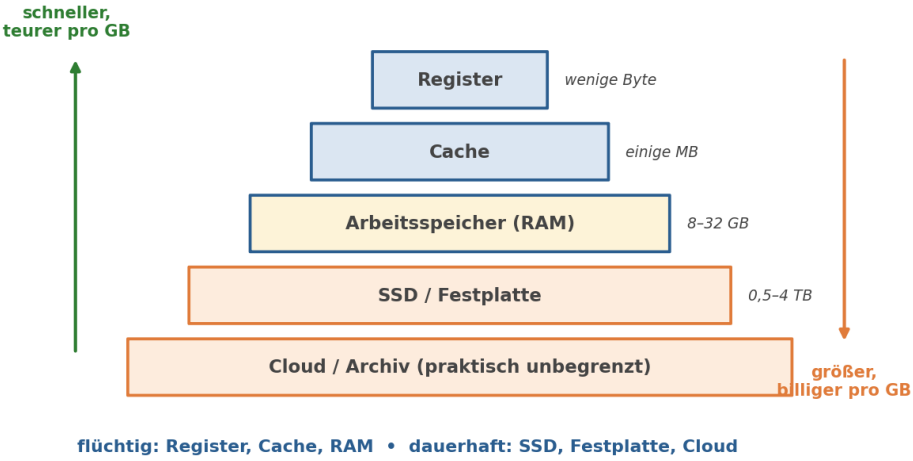


Abbildung 4: Speicherhierarchie

4. Flüchtig oder dauerhaft?

Die Stromausfall-Frage: Was überlebt, wenn der Stecker gezogen wird?

- **Flüchtig** (weg bei Stromverlust): Register, Cache, **RAM**
- **Dauerhaft** (bleibt erhalten): **SSD, Festplatte**, USB-Stick, Cloud

Die Veggie-Soles-Bestellung ist erst dann **sicher**, wenn sie vom RAM auf die SSD geschrieben wurde. Genau dieses “dauerhaft machen” begegnet Ihnen beim Speichern von Dateien (Notebook 12) und Datenbanken (Notebook 22) wieder.

4. Live-Demo: Speicher im Blick

▶ **Live-Demo** – Speicher bei der Arbeit zusehen:

1. **Aktivitätsmonitor** (macOS) bzw. **Task-Manager** (Windows): Wie viel RAM ist belegt? Welches Programm braucht am meisten?
2. Python als Messgerät (wieder nur Taschenrechner-Modus):

```
import sys
sys.getsizeof(42)        # 28 Byte
sys.getsizeof("Veggie")  # 55 Byte
```

Selbst eine kleine Zahl belegt Speicher – Text entsprechend mehr.

5. Das Betriebssystem: der Verwalter

Windows, macOS, Linux, Android, iOS – alle lösen dieselben vier Aufgaben:

Aufgabe	Bedeutung
Prozessverwaltung	teilt die CPU zwischen laufenden Programmen auf
Speicherverwaltung	weist jedem Programm seinen RAM-Bereich zu
Dateiverwaltung	organisiert Ordner und Dateien auf SSD/Festplatte
Geräteverwaltung	vermittelt zu Tastatur, Drucker, Netzwerk (Treiber)

Ihr Programm redet **nie direkt** mit der Hardware – es stellt Anfragen ans Betriebssystem.

5. Sofort ausprobieren: Prozesse auf Ihrem Gerät

✎ **Sofort ausprobieren** (2 Minuten, eigenes Gerät):

1. Öffnen Sie den **Task-Manager** (Windows: `Strg+Umschalt+Esc`) bzw. **Aktivitätsmonitor** (macOS: `Cmd+Leertaste`, “Aktivitätsanzeige”).
2. Wie viele Prozesse laufen gerade? Hätten Sie so viele erwartet?
3. Welche drei Programme belegen am meisten **Arbeitsspeicher**?

5. Vom Python-Programm zur Hardware

Die Schichten, durch die Ihr Code ab nächster Woche läuft:

Schicht	Beispiel
Ihr Programm	`print("Hallo Veggie Soles")`
Python-Interpreter	übersetzt in Maschinenbefehle (Notebook 01)
Betriebssystem	teilt CPU-Zeit und Speicher zu
Hardware	CPU führt aus, Transistoren schalten

Jede Schicht **versteckt** die Komplexität darunter. Deshalb können Sie programmieren, ohne Transistoren zu verdrahten.

Synthese: Eine Bestellung bei Veggie Soles

Kundin klickt **“Bestellen”** – was passiert?

1. **Eingabe:** Klick kommt über Netzwerk/Tastatur ins System
2. Betriebssystem reicht die Daten an den Shop-Prozess weiter
3. Programm + Bestelldaten liegen im **RAM**; die **CPU** holt Befehl für Befehl: Preis addieren, Lagerbestand vergleichen (ALU!)
4. Ergebnis wird auf die **SSD** geschrieben – jetzt ist die Bestellung **dauerhaft**
5. **Ausgabe:** Bestellbestätigung erscheint auf dem Monitor

Ein Klick – und **jede Ebene von heute** war beteiligt: Transistoren, Binärzahlen, von Neumann, Speicherhierarchie, Betriebssystem.

Zusammenfassung: Heute gelernt

- **Transistor → Gatter → Rechenwerk:** Computer rechnen mit an/aus-Schaltern
- **Bit & Byte:** 8 Bit = 256 Zustände; binär ↔ dezimal ↔ hex umrechnen können Sie jetzt von Hand
- **von Neumann:** Programm + Daten im selben Speicher; Steuerwerk, ALU, Register; Holen–Dekodieren–Ausführen
- **Speicherhierarchie:** Register bis Cloud; flüchtig (RAM) vs. dauerhaft (SSD)
- **Betriebssystem:** verwaltet Prozesse, Speicher, Dateien, Geräte

Ausblick

- **Nächste Woche:** Notebook 01 – Programmiersprachen. Wie kommt Ihr Python-Code durch den Interpreter in die Maschine, die Sie heute kennengelernt haben?
- Die Theorie von heute taucht wieder auf:
  - **Foliensatz 05:** Zeichen als Zahlen (ASCII/Unicode), Gleitkommazahlen
  - **Foliensatz 08:** UND/ODER/NICHT als Python-Operatoren
  - **Notebook 12 & 22:** Daten dauerhaft speichern (Dateien, Datenbanken)

Bis dahin: Python installieren (Anleitung in Moodle) – nächste Woche wird getippt.